Claude Code使用感の報告会（2026年3月）

パーソルクロステクノロジー株式会社
エンジニアリング事業管掌　設計統括本部
第2電子・制御設計本部　第1設計部　5G　阿部耕二

目次

- 自己紹介
- Claude Codeを使うためにかかるコスト
- 環境構築
- 事例紹介1: チャット Webアプリ
- 事例紹介2: 営業日報システム Webアプリ
- 事例紹介3: 既存プロダクト（組込みソフトウェア）の分析
- Claude Codeを使って嬉しかったこと
- Claude Codeを使って困ったこと
- Claude Codeの使い方をある程度覚えた今、思っていること
- 参考資料

自己紹介

- 名前: 阿部　耕二（あべ　こうじ）
- 所属: パーソルクロステクノロジー株式会社
  エンジニアリング事業管掌　設計統括本部
  第2電子・制御設計本部　第1設計部　5G
- 医療機器の組込みソフトウェア開発。C言語。
- LAPRASポートフォリオ: https://lapras.com/public/k-abe
- X: @juraruming

Claude Codeを使うためにかかるコスト

チーム、エンタープライズ

Claude

claude.ai/upgrade

成長に合わせて拡張できるプラン

個人 | TeamおよびEnterprise

チーム

5〜150ユーザー

シートあたりの予測可能な使用量

Standardシート USD 20 /月

Claudeの全機能に加え、Proより多くの使用量*
月額請求の場合 USD 25/月

Premiumシート USD 100 /月

標準シートの5倍の使用量*
月額請求の場合 USD 125/月

- 200Kコンテキストウィンドウ
- 追加使用分はAPI料金で利用可能
- Claude Code
- Cowork
- 一括請求と管理
- シングルサインオン（SSO）とドメイン管理
- リモートおよびローカルコネクタの管理者コントロール
- Claudeデスクトップアプリのエンタープライズ展開
- 組織全体のエンタープライズ検索
- Microsoft 365、Slackなどと連携
- デフォルトでお客様のコンテンツをモデルトレーニングに使用しません

Teamプランを取得

仕事用メールアドレスが必要です。

エンタープライズ

20人以上のユーザー

プール型の柔軟な使用量

シート料金 + API料金での使用量

USD 20/シート。使用料金はモデルとタスクに応じて変動します。

Teamのすべての機能に加えて：

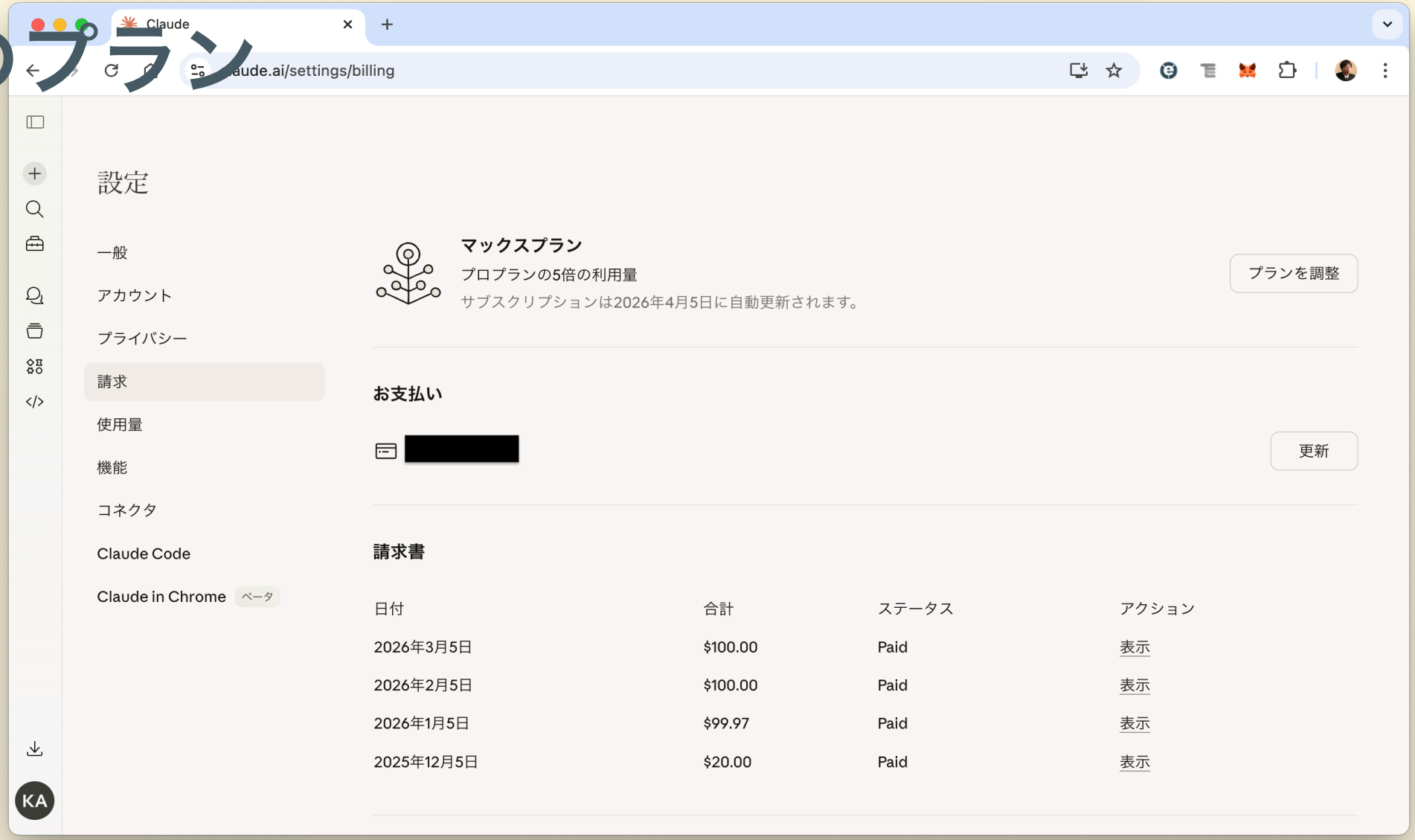
- 組織全体でプールされた使用量による従量課金制
- ユーザーと組織の利用上限を設定
- 500Kコンテキストウィンドウ
- きめ細かい権限設定によるロールベースアクセス
- クロスドメインID管理システム（SCIM）
- 監査ログ
- 可観測性と監視のためのCompliance API
- ネットワークレベルのアクセス制御
- カスタムデータ保持コントロール
- IP許可リスト
- Google Docsの整理機能

Enterpriseプランを取得

Enterpriseアカウントを作成するには、仕事用メールアドレスが必要です。 詳細については営業担当にお問い合わせください。

*利用制限が適用されます。表示されている価格には該当する税金は含まれていません。

私のプラン


Claude
aude.ai/settings/billing
設定
一般
アカウント
プライバシー
請求
使用量
機能
コネクタ
Claude Code
Claude in Chrome ベータ
KA
マックスプラン
プロプランの5倍の利用量
サブスクリプションは2026年4月5日に自動更新されます。
プランを調整
お支払い
[redacted]
更新
請求書
日付 合計 ステータス アクション
2026年3月5日 $100.00 Paid 表示
2026年2月5日 $100.00 Paid 表示
2026年1月5日 $99.97 Paid 表示
2025年12月5日 $20.00 Paid 表示

環境構築

- Claude Codeと**ghコマンド**の連携は是非ともオススメ
- リポジトリ作成、Issueの作成、プルリクエスト、レビューなどがターミナルで完結します。
- 圧倒的開発体験が得られて感動しました。

事例紹介1: チャット Webアプリ

- 参考資料1. Claude CodeによるAI駆動開発入門を写経
- GitHubリポジトリ：
  https://github.com/grace2riku/ai-chat

試したこと

■MCP
Claude Codeと他のアプリケーションとのやりとりを仲介してくれる通信規格

- Context7：ライブラリの最新のドキュメントを参照してくれる
- Playwright：ブラウザを操作してテストを自動化する

■要件定義

- Claude Codeと共同して行い、CLAUDE.mdにまとめる
- プロンプト：

```
> AIチャットボットを作りたいです。仕様書をあなたと一緒に作りたいので、必要な情報は質問してもらえますか?最後に CLAUDE.md ファイルを出力するのが目的です。
```

- CLAUDE.md
  https://github.com/grace2riku/ai-chat/blob/main/CLAUDE.md

■タスクの管理

- 要件定義した内容を実現するための実行計画を立てる。
  - TODO.mdが作成された。チェックボックス付き
- プロンプト：

```
現在のアプリケーションを構築するための実行計画を立ててください。それをTODOリストとしてMarkDownファイルに落としてください。
```

- TODO.md
  https://github.com/grace2riku/ai-chat/blob/main/TODO.md

■実装

- TODO.mdを元に実装してもらう。
- プロンプト：

```
@TODO.md これを元にフェーズ1の実装をお願いします。終わったらチェックボックスにチェックをしてください。use context7
```

- ここでuse context7を使っている。CLAUDE.mdでNext.jsを使うと書いている。Next.jsのドキュメントを参照して実装してくれる（らしいです）。

事例紹介2: 営業日報システム Webアプリ

- 参考資料1. Claude CodeによるAI駆動開発入門を写経
- GitHubリポジトリ：
  https://github.com/grace2riku/daily-report

試したこと

■要件定義をCLAUDE.md に書いてもらう。

- 設計書（画面設計、テスト、その他）も書いてもらい、CLAUDE.mdから参照する。
- 要件を実現するための使用技術をCLAUDE.mdに追加した。
- CLAUDE.md https://github.com/grace2riku/daily-report/blob/main/CLAUDE.md

■ガードレールの用意

- **ガードレール**の意味
  - AIが期待しない動作をしないように準備しておく
- 守るべき仕様や規約を与える。
  - 例）コードの命名規則の統一
  - 例）コードの記法のルールの統一
- リンター(ESLint)の設定をいれた。
- Huskyを入れてGitコミットやプッシュの前にリンターを実行させるようにした。

■CI/CDの設定

- Claude Codeに実装を書いてもらう→プッシュ前にリンター・テストを実行→デプロイ、の流れを作成してもらった。

■GitHubのIssueを作成してもらう

- プロンプト：

```
現在の仕様と構成に従ってまずIssueを細かく具体的に立ててください。それをghコマンドでGitHubにデプロイしてください。ultrathink
```

- 「細かく具体的に」の指示が個人的に重要だと思っている。
  - 大きい粒度のIssueにするとAIの成果物も安定しない（ような気がします）。
- ultrathinkはよく考えて精度の高い回答を行う。トークンを多く消費する。

作成されたIssue。ラベル（frontend, backend, その他）もつけてくれてありがたい。

Issues · grace2riku/daily-repo

github.com/grace2riku/daily-report/issues?q=is%3Aissue%20state%3Aclosed&page=1

更新を完了

- 単体テスト - カスタムフック priority:medium test
  #47 · by grace2riku was closed on Jan 8 — 1
- 単体テスト - ユーティリティ関数 priority:medium test
  #46 · by grace2riku was closed on Jan 8 — 1
- [Epic] テストの実装 epic priority:medium test
  #45 · by grace2riku was closed on Jan 9 — 1
- 顧客マスタ画面 (SCR-006) の実装 frontend master priority:medium
  #44 · by grace2riku was closed on Jan 8 — 1
- DELETE /api/v1/customers/{id} の実装 backend master priority:medium
  #43 · by grace2riku was closed on Jan 8 — 1
- PUT /api/v1/customers/{id} の実装 backend master priority:medium
  #42 · by grace2riku was closed on Jan 8 — 1
- GET /api/v1/customers/{id} の実装 backend master priority:medium
  #41 · by grace2riku was closed on Jan 8 — 1
- POST /api/v1/customers の実装 backend master priority:medium
  #40 · by grace2riku was closed on Jan 8 — 1
- GET /api/v1/customers の実装 backend master priority:medium
  #39 · by grace2riku was closed on Jan 8 — 1
- [Epic] 顧客マスタ管理機能の実装 epic master priority:medium
  #38 · by grace2riku was closed on Jan 9 — 1

■実装

- 作成したIssueを指定し実装を指示する。
- gitを使った変更管理を意識した指示をする。
- 「ブランチを切って作業し、最後にプルリクエストしてください。」を入れないとメインブランチで作業する。コミットしてもプルリクエストまで出してくれなかったりする。
- プロンプト：

```
Issue #1を実装してください。use context7
ブランチを切って作業し、最後にプルリクエストしてください。
```

■レビュー

- プルリクエストをレビューする。
- プロンプト：

```
PR  #1の内容をレビューしてください。
プルリクエストのコメントとしてレビュー内容を記述してください。
```

- レビューの一例
  https://github.com/grace2riku/daily-report/pull/56#pullrequestreview-3618259265

■GitWorktreeによる並行処理

- GitWorktreeは同じGitリポジトリの別ブランチを別ディレクトリにチェックアウトできる。
- フロントエンドのIssueの#xはターミナル1で実装する。バックエンドのIssue #yはターミナル2で並行して行う、みたいなことができる。

- 例1-1) ターミナル1でClaude Codeを起動し、Issue #2（バックエンド）のIssueを対応する。
- プロンプト：

```
Issue #2を実装してください。別ブランチを立てて、最後プルリクエストをお願いします。
```

- 例1-2) ターミナル2でClaude Codeを起動し、Issue #3（フロントエンド）のIssueを対応する。
- プロンプト：

```
Issue #3を実装してください。git worktreeを使ってサブフォルダを作り別ブランチを立てて、最後プルリクエストをお願いします。
サブフォルダは`issue-***'という命名規則に従ってください。
```

- フロントエンドのIssue、バックエンドのIssueを並行で実装させたのは、競合が起きにくいだろう、という考えからそうしている。
- GitWorktreeによる並行処理でIssueは早く対応できるようになったが、Proプランの使用制限にひっかかり5時間待たされることが多くなってきた。
  Pro→Maxプランへ切り替え、その後は使用制限がかかることはなくなった。現在も快適に使えている。

■サブエージェント

- サブエージェント == その道の専門家
- サブエージェントはコンテキストを共有しない。
- 今回はフロントエンド、バックエンド、インフラ、テスト、のサブエージェントを作成した。
- /agents コマンドでエージェントを作成できる。
- 作成したサブエージェントは.claude/agentsディレクトリ以下に保存される。
  https://github.com/grace2riku/daily-report/tree/main/.claude/agents

- Issueにラベルがついていれば、以下のプロンプトで指示すると最適なサブエージェントで実装をおこなってくれる。

```
Issue #44 このIssueに最適なサブエージェントで実装してください。
```

- Issue #44はフロントエンドの対応だった。上の場合はフロントエンドのサブエージェントが実装を担当してくれる。

■カスタムスラッシュコマンド

- Issueの対応で想定する動作
  - Issueごとにメインブランチからブランチ作成→実装→コミット→プルリクエスト→メインブランチにマージと同時にIssueクローズ→メインブランチに戻る
    想定どおりに動いてくれないときがある。カスタムスラッシュコマンドで複数のコマンドをまとめておく。
- 「なんか毎回同じプロンプトを指示しているぞ」と感じた時にカスタムスラッシュコマンド化すると便利だと思う

今回つくったカスタムスラッシュコマンド

- .claude/commands/implement-issue-worktree-subagent.md
  イシューの一連の対応
- .claude/commands/pr-review.md
  プルリクエストのレビュー
- コマンド実行例）Issue #44の対応

```
/implement-issue-worktree-subagent 44
```

コマンド実行例）Issue #44のプルリクエスト#92をレビュー

```
/pr-review 92
```

■Agent Skills

- デプロイ前のチェックを行うスキルをつくった。
  - .claude/skills/pre-deploy-check/SKILL.md

- 以下を理解できていなかったのでClaudeに聞きました。
  - MCP、サブエージェント、Agent Skillsの違い・使い分けがいまいち理解できていない
  - カスタムスラッシュコマンドとAgent Skillsの違い・使い分けがいまいち理解できていない

https://claude.ai/share/72748cc4-9506-4c36-b651-82a8d189258f

- 上記リンクの説明で個人的に**カスタムスラッシュコマンドはユーザーが意識的に使いたい時**に使う、**Agent SkillsはClaudeに自律判断で使わせたいとき**に使う、という説明で使い分け方が腑に落ちました。

事例紹介3: 既存プロダクト（組込みソフトウェア）の分析

- 事例紹介1、2はWebアプリケーションの開発
- 私の専門領域は組込みソフトウェア。組込みソフトウェアでClaude Codeを使ってみたいと思った。
  - マイコン評価ボードのサンプルプログラムをClaude Codeで分析してみることにした。

- 分析対象ハードウェア：Renesasエレクトロニクス社製マイコン評価ボード　EK-RA8P1
- 分析対象ソフトウェア：ra-fsp-examplesのquickstart_ek_ra8p1_ep
  - LEDの制御、カメラ撮影画像をLCD表示、LCDタッチなど、クイックスタート的なサンプルプログラム
- 分析結果GitHubリポジトリ：ra-fsp-examples_ek_ra8p1_analysis
- 分析した背景・理由：
  - 対象ハードウェアでソフトウェア開発コンテストに出場予定
  - 対象ハードウェアに搭載されているマイコンを使ったことがない
    ➡Claude Codeでサンプルコードを分析してみよう!!!

- Q. 分析の結果はどうだったか?
  - A. 想像以上の分析結果を得られた。
    組込みソフトウェアはソフトウェアだけでなく、マイコンのハードウェア的な要素もあるため、Webアプリのような精度の高い分析はできないだろうな・・・と思っていたが、予想を超える分析結果だった。
- 分析結果
  - 分析結果のレポートはこちらのリンクにマークダウンで出力している
  - 17個の観点からレポートを出力してもらった

- どのように分析したのか?
  - 分析してほしい観点をマークダウン形式でファイルに書く
  - マークダウンを指定し分析を依頼する。以下のようなプロンプトで実行した。
  - doc/analysis_request/phase01_file_structure.mdは分析してほしい観点が書いてあるファイル

```
doc/analysis_request/phase01_file_structure.md を読んで解析を実行してください
```

Claude Codeを使って嬉しかったこと

1. 開発が楽しい
2. gitコミットログを考えて書かなくなり楽になったこと
3. イシューの作成が楽になったこと
4. GitHubリポジトリのプロダクトなどを分析、レポート化し自分の知らない技術を知れること

Claude Codeを使って困ったこと

1. Claude Codeを使うことが目的になっている時があること

- Claude Codeを使って価値あるソフトウェアを作ることが主目的のハズ!

2. 期待するアウトプットが得られない時があること
3. Claude Codeのアップデートが早すぎて、新機能のキャッチアップができなくて焦ること

Claude Codeの使い方をある程度覚えた今、思っていること

1. エージェント型コーディングAIの使い方ではなく、AIが正しく動く仕組み作りが重要ではないかと思っていること

- Harness engineering（ハーネスエンジニアリング）が気になっている・・・。

参考資料

1. Claude CodeによるAI駆動開発入門
2. 実践Claude Code入門
   –現場で活用するためのAIコーディングの思考法

ご清聴ありがとうございました🙏